

Modificadores Avanzados de Eventos en Vue 3

Propagación, comportamiento por defecto, captura, rendimiento y atajos.



El ADN de los Eventos en Vue

En Vue, `@evento` es una abstracción declarativa para escuchar eventos nativos del navegador.

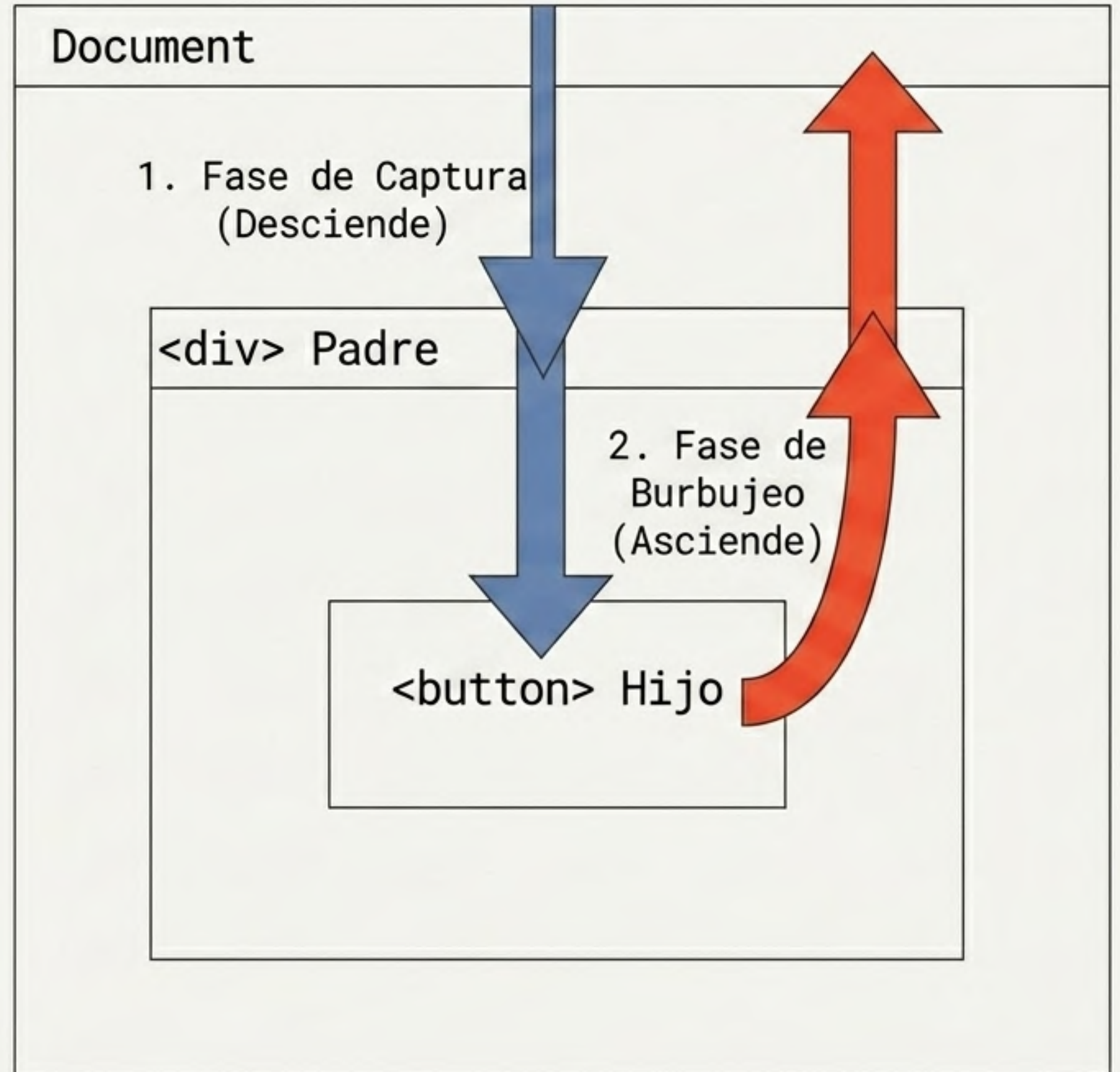
Los modificadores son sufijos que alteran cómo y cuándo se ejecuta tu código, manteniendo la lógica limpia y separada de las mecánicas del DOM.

```
<!-- Sin modificador -->
<button @click="saludar">Saludar</button>

<!-- Con modificadores avanzados -->
<button @click.stop="saludar">No propagar</button>
<form @submit.prevent="enviar">No recargar</form>
<input @keydown.ctrl.k="buscar">
```


Anatomía de la Propagación

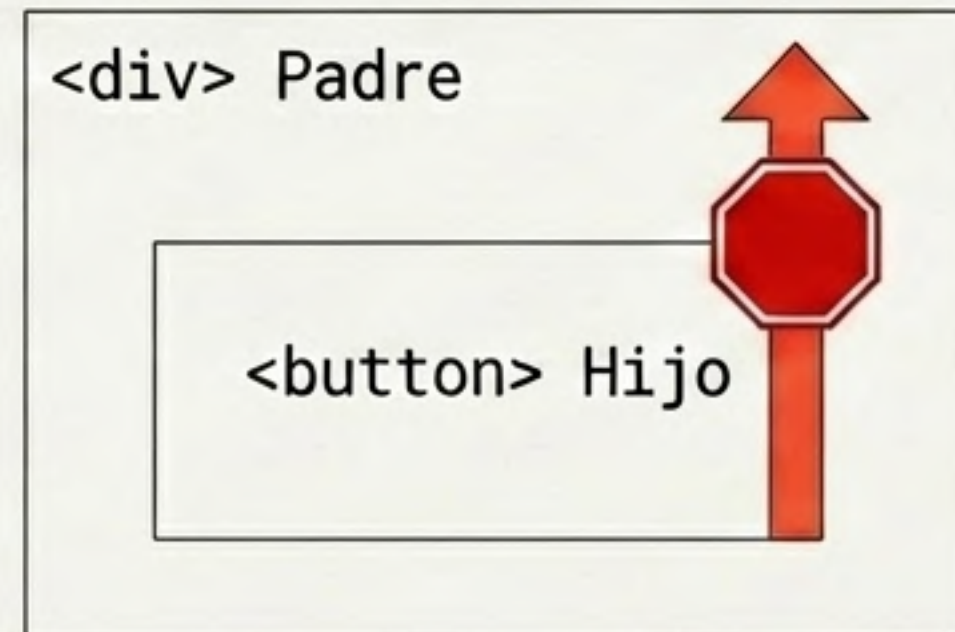
Cuando ocurre un evento, no solo afecta a su objetivo. Viaja a través de sus ancestros en dos fases:



Detener el Ascenso: .stop

El modificador `.stop`` es el equivalente declarativo a `event.stopPropagation()`. Detiene el burbujeo inmediatamente. El evento se ejecuta en el elemento actual, pero nunca alcanza a los elementos padre.

```
<div @click="seleccionarTarjeta">  
  <!-- El clic activa el botón, pero el  
    evento se detiene aquí y no sigue  
    propagándose hacia el padre. -->  
  <button @click.stop="marcarFavorito">  
    Favorito  
  </button>  
</div>
```



Caso de Uso: .stop

Tarjeta interactiva con acciones secundarias anidadas.

```
<script setup>
import { ref } from 'vue'

const tarjetaSeleccionada = ref(false)
const esFavorito = ref(false)
const toggleTarjeta = () => tarjetaSeleccionada.value = !tarjetaSeleccionada.value
const toggleFavorito = () => esFavorito.value = !esFavorito.value
</script>

<template>
  <div class="tarjeta" @click="toggleTarjeta">
    <p>Seleccionada: {{ tarjetaSeleccionada }}</p>
    <!-- El clic altera esFavorito, pero NO altera tarjetaSeleccionada -->
    <button @click.stop="toggleFavorito">★ Favorito</button>
  </div>
</template>
```

Protege al padre
de recibir el clic

Vetar al Navegador: .prevent

.prevent es el equivalente a `event.preventDefault()`. Impide que el navegador ejecute su acción predeterminada (recargar o navegar), pero NO detiene la propagación del evento en el DOM.



Sin recarga

```
<!-- Evita que la página se recargue al enviar -->  
<form @submit.prevent="enviarFormulario">  
  <button type="submit">Enviar</button>  
</form>
```



Sin navegación

```
<!-- Evita que el navegador abra el enlace -->  
<a href="https://ejemplo.com" @click.prevent="abrirModal">  
  Ver más  
</a>
```

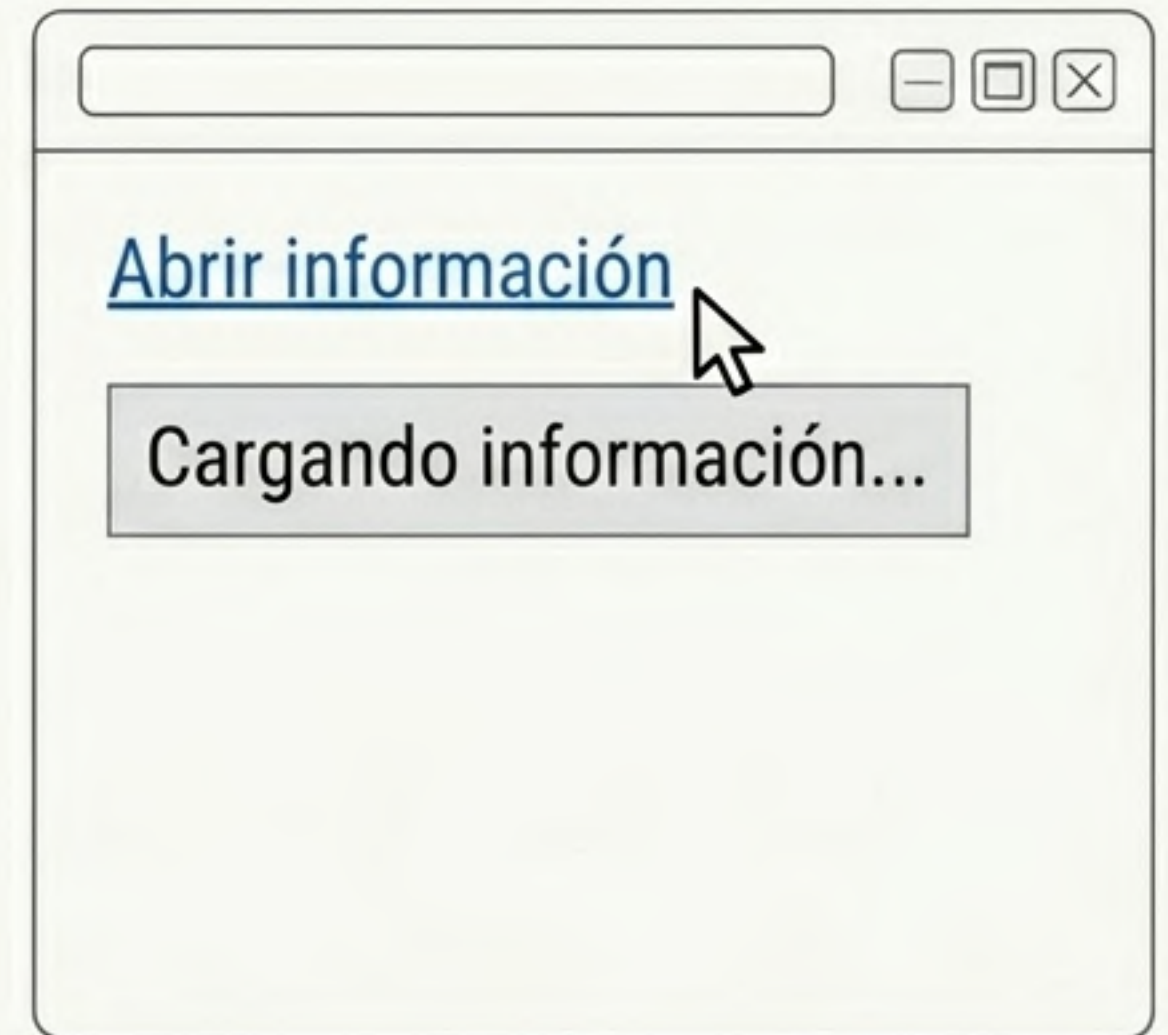

Caso de Uso: .prevent

Mantener la accesibilidad web usando un enlace real (<a>), pero manejando la carga de datos internamente con Vue.

```
<script setup>
import { ref } from 'vue'

const mensajeInfo = ref('')
const mostrarMensaje = () => {
  mensajeInfo.value = 'Cargando información...'
}
</script>

<template>
  <a href="/ayuda" @click.prevent="mostrarMensaje">
    Abrir información
  </a>
  <p v-if="mensajeInfo">{{ mensajeInfo }}</p>
</template>
```



El Poder del Encadenamiento

Los modificadores se pueden combinar. Nota clave: Vue los procesa exactamente en el orden en que están escritos.

```
<a href="/info" @click.stop.prevent="abrirInfo">
```

1. Detiene el burbujeo hacia el elemento padre.

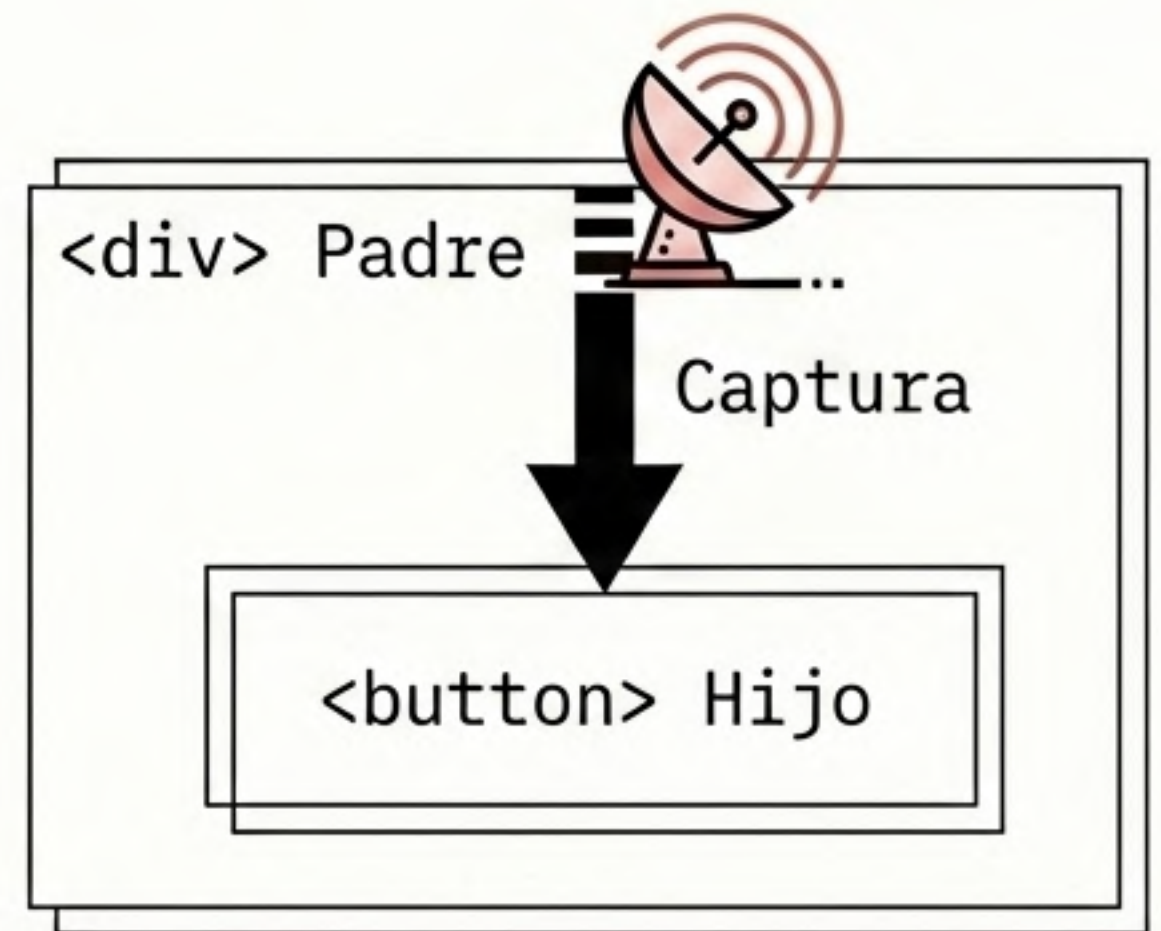
2. Evita la navegación predeterminada del navegador.

3. Ejecuta tu función manejadora.

Invertir la Gravedad: .capture

.capture obliga al manejador a ejecutarse en la fase de captura, es decir, de afuera hacia adentro. El padre intercepta el evento antes de que llegue al hijo.

```
<!-- Se ejecuta PRIMERO, antes de que el clic  
llegue al botón -->  
<div @click.capture="auditarClicPadre">  
  <button @click="clicHijo">Comprar</button>  
</div>
```



Caso de Uso: .capture

Control estricto del orden de ejecución.

```
<script setup>
import { ref } from 'vue'
const registro = ref([])
const log = (msg) => registro.value.push(msg)
</script>
<template>
  <div @click.capture="log('1. Captura Padre')">
    <button @click="log('2. Clic Hijo')">Haz Clic</button>
  </div>
</template>
```

Registro de Interfaz

- > 1. Captura Padre
- > 2. Clic Hijo

La Promesa de Rendimiento: .passive

Mejor rendimiento en scroll/touch: En eventos intensivos (**scroll**, **touch**, **wheel**), el navegador se detiene para comprobar si usarás **preventDefault()**, causando tirones (jank). **.passive** es una promesa: "No usaré preventDefault, haz el scroll inmediatamente".
Regla de oro: Nunca combines **.passive** con **.prevent**'.

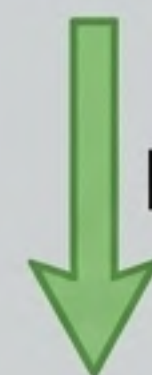
Sin .passive (Bloquea)



Jank

Bloqueo del Scroll

Con .passive (Fluido)



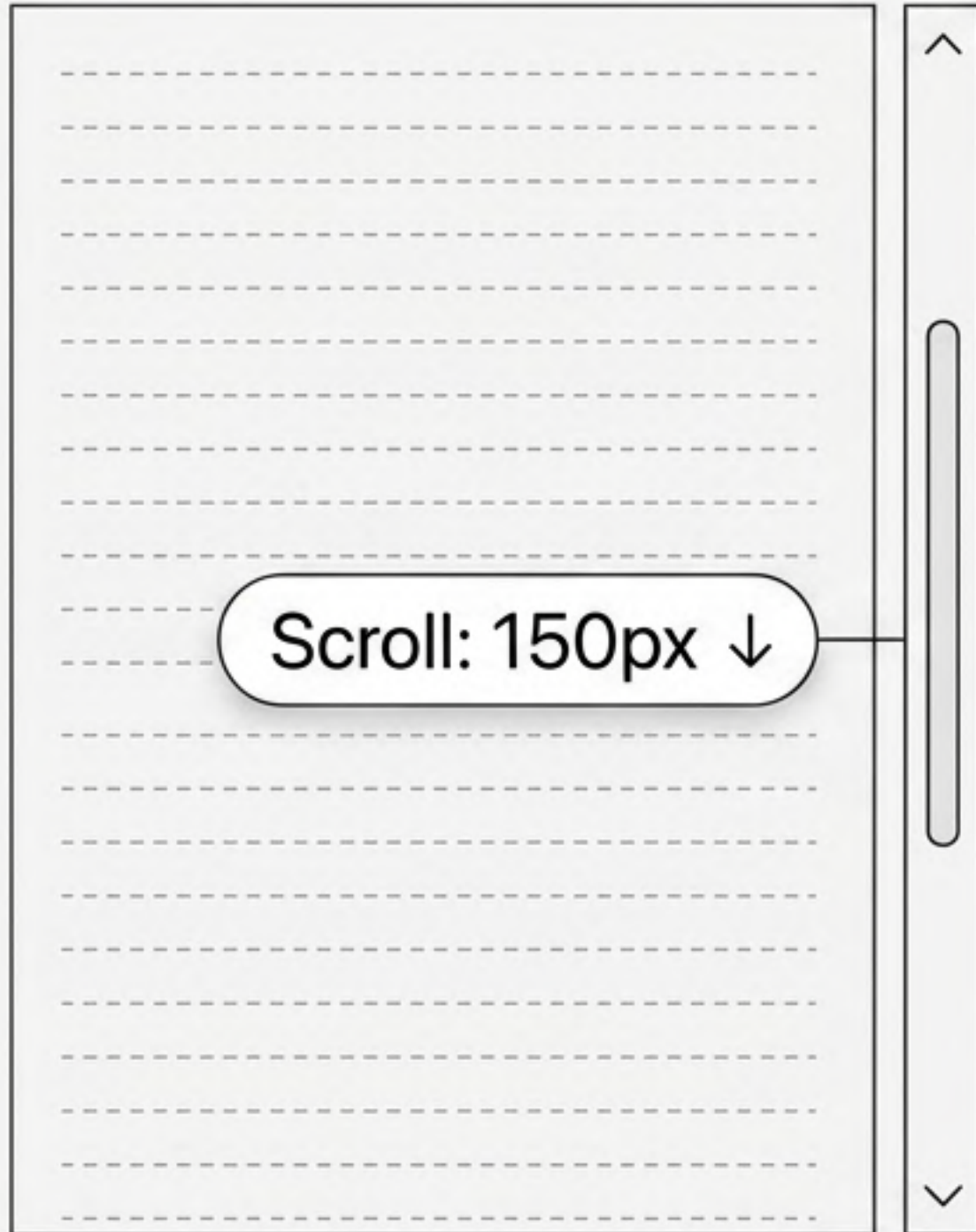
Fluido

Scroll Inmediato

```
<div @scroll.passive="alHacerScroll">...</div>  
.passive: Promesa de no bloqueo
```


Caso de Uso: .passive

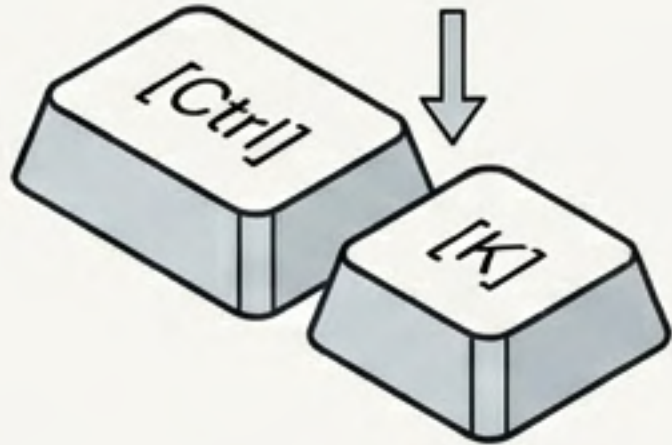
Monitorizar el scroll del usuario sin sacrificar los FPS (fotogramas por segundo).



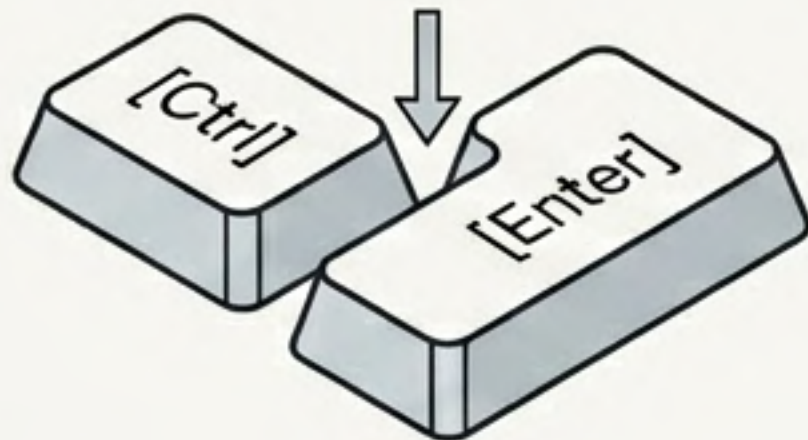
```
1  <script setup>
2  import { ref } from 'vue'
3  const scrollY = ref(0)
4
5  const onScroll = (evento) => {
6    scrollY.value = evento.target.scrollTop
7  }
8  </script>
9
10 <template>
11   <div class="contenedor" @scroll.passive="onScroll">
12     <p v-for="n in 30" :key="n">Línea {{ n }}</p>
13   </div>
14   <p>Scroll: {{ scrollY }}px</p>
15 </template>
```


Atajos Maestros: .ctrl.k

Vue permite mapear eventos combinando modificadores de sistema (.ctrl`, .alt`, .shift`, .meta`) con teclas específicas. El evento solo se dispara si todas las condiciones se cumplen simultáneamente.



```
<!-- Requiere Control + K simultáneamente -->  
<input @keydown.ctrl.k.prevent="abrirBuscador">
```



```
<!-- Requiere Control + Enter simultáneamente -->  
<textarea @keydown.ctrl.enter="enviarMensaje"></textarea>
```


Caso de Uso: Ctrl + K

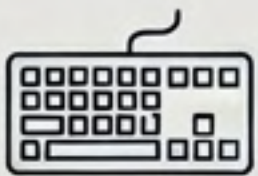
Crear el clásico atajo para abrir un buscador universal (paleta de comandos). Se usa `.prevent` para evitar que el navegador enfoque su propia barra de búsqueda.

```
1 <script setup>
2 import { ref } from 'vue'
3 const buscadorAbierto = ref(false)
4 const abrirBuscador = () => buscadorAbierto.value = true
5 </script>
6 <template>
7   <!-- tabindex="0" permite que el div reciba eventos de teclado -->
8   <div tabindex="0" @keydown.ctrl.k.prevent="abrirBuscador">
9     <p>Presiona Ctrl + K para buscar</p>
10    <div v-if="buscadorAbierto" class="modal">
11      Buscador abierto 🔍
12    </div>
13  </div>
14 </template>
```

Detalle técnico vital para divs

Cheat Sheet: Modificadores Vue 3

Evita abusar de los modificadores; úsalos con precisión. Cambia el estado reactivo (ref) y deja que Vue actualice la UI.

	.stop	¿Qué hace? Detiene burbujeo.	¿Uso ideal? Proteger al padre de clics en el hijo.
	.prevent	¿Qué hace? Anula acción nativa.	¿Uso ideal? Formularios y Links dinámicos.
	.capture	¿Qué hace? Intercepta bajando.	¿Uso ideal? Controlar el evento antes que el hijo.
	.passive	¿Qué hace? Promete no bloquear.	mejor rendimiento en scroll/touch
	.enter / .esc / .ctrl.k	¿Qué hace? Filtra combinación.	¿Uso ideal? Atajos de teclado precisos.